

AST

Outline

멘토님이 target.c 라는 C 코드를 미리 분석해서 ast.json 으로 주셨다.

내가 할 일은 `ast.json` 을 읽어서 함수 개수, 함수 이름, 반환 타입, 매개변수 개수, if문 개수를 자동으로 추출하는 `ast.c` 프로그램을 만드는 것이다.

즉, 이미 만들어진 `ast.json` 파일을 C 코드로 읽고 필요한 정보만 추출하는 과정이다.

한마디로 정리된 결과(JSON)를 C 프로그램으로 다시 읽어 필요한 정보를 추출하는 과정이다.

- **파싱(Parsing)** : 사람이 읽기 어려운 데이터를 프로그램이 읽어서 필요한 정보만 추출하는 과정

`ast.json` 파일은 5,000줄이 넘기 때문에 사람이 직접 확인하기 어렵다. 그래서 C 프로그램이 파일을 읽어 필요한 정보만 찾아 출력하도록 구현했다.

JSON

JSON은 "이름표 : 값" 형태로 데이터를 저장하는 방식이다.

주로 사용하는 기호는 두 가지이다.

- `{ }` : 이름표와 값이 들어 있는 객체(Object)
- `[ ]` : 여러 개의 데이터를 순서대로 저장하는 배열(Array)

`ast.json` 파일은 객체 안에 또 다른 객체나 배열이 계속 들어 있는 중첩 구조로 되어 있다. 그래서 원하는 정보를 찾으려면 안쪽으로 차례대로 들어가면서 값을 확인해야 한다.

EX_1 : ast.json의 malloc

멘토님이 제공한 `ast.json` 파일의 일부를 보면 다음과 같은 구조로 되어 있다.

```
{
  "_nodetype": "Decl",
  "name": "malloc",
  "type": {
    "_nodetype": "FuncDecl",
```

```

    "args": {
      "_nodetype": "ParamList",
      "params": [ ... ]
    }
  }
}

```

1단계 - 가장 바깥 객체

```

{ "_nodetype": "Decl", "name": "malloc", "type": {...} }

```

- `Decl` 은 선언(Declaration)을 의미한다.
- 함수 이름은 `malloc` 이다.
- 함수에 대한 자세한 정보는 `type` 안에 들어 있다.

2단계 - type 확인

```

"type": { "_nodetype": "FuncDecl", "args": {...} }

```

- `FuncDecl` 이므로 함수 선언임을 알 수 있다.
- 매개변수 정보는 `args` 안에 저장되어 있다.

3단계 - args 확인

```

"args": { "_nodetype": "ParamList", "params": [...] }

```

- `params` 배열 안에 실제 매개변수 목록이 들어 있다.

※ `malloc` 의 매개변수를 확인하려면 `Decl → type → args → params` 순서로 안쪽 구조를 따라가야 한다.

```

Decl (malloc 선언)
├─ type: FuncDecl (함수)
│   └─ args: ParamList (인자 목록 상자)
│       └─ params: [파라미터1, 파라미터2, ...]

```

추출 목록

추출 정보	위치	설명
함수 이름	decl.name	함수 이름이 저장되어 있는 필드이다.
리턴 타입	decl.type.type부터 시작	반환 타입은 여러 단계로 감싸져 있어서 <code>PtrDecl</code> , <code>TypeDecl</code> 등을 따라가다 보면 <code>IdentifierType</code> 에서 실제 타입 이름을 확인할 수 있다.
파라미터	decl.type.args.params	<code>params</code> 배열에 함수의 매개변수 정보가 들어 있다.
if문 개수	함수 내부 전체 탐색	<code>If</code> 노드가 여러 코드 블록 안에 중첩될 수 있기 때문에 함수 내부를 재귀적으로 탐색하면서 개수를 센다.

json_c.c 에서 실제 사용한 함수

함수	용도
json_create(문자열)	JSON 문자열을 파싱해서 사용할 수 있는 형태로 변환한다.
json_get(v, "키")	객체에서 키를 이용해 값을 가져온다.
json_get(v, 숫자)	배열에서 해당 순서의 값을 가져온다.
json_get(v, "a", "b", 0)	여러 단계를 한 번에 따라가 원하는 값을 가져온다.
json_get_string(v, "키")	가져온 값을 문자열로 변환한다.
json_len(v)	배열이나 객체의 개수를 확인한다.
v.type == JSON_OBJECT	현재 값이 객체인지 확인한다.
json_object 구조체의 keys[], values[]	객체 안의 모든 항목을 순회할 때 사용했다. if 문 개수를 세는 재귀 탐색에서 활용했다.

사용하지 않은 함수

이번 과제에서는 아래 함수들은 사용하지 않았다.

- json_string_to_value
- json_create_array
- json_create_object
- json_stacktrace_*
- json_fprint_*
- json_free*

직접 구현한 함수

`json_c.c` 헤더를 보면 다음과 같이 `json_read()` 가 선언만 되어 있고 실제 구현(몸통)이 없는 상태였다.

```
//TODO read json file
json_value json_read(const char * const path);
```

즉 `json_read("ast.json")` 을 호출해도 실제로 하는 일이 없어서 사용할 수 없었다. 파일을 열어서 문자열로 만들어주는 부분을 직접 구현해야 했으며, 이를 위해 `my_json_read()` 함수를 새로 작성하여 사용하였다.

```
json_value my_json_read(const char *path) {
    FILE *fp = fopen(path, "rb");           // 1. 파일 열기
    fseek(fp, 0, SEEK_END);
    long size = ftell(fp);                 // 2. 파일 전체 크기 계산
    fseek(fp, 0, SEEK_SET);
    char *buffer = (char *)malloc(size + 1);
    fread(buffer, 1, size, fp);            // 3. 파일 내용을 전체를 문자열로 읽기
    buffer[size] = '\0';
    fclose(fp);
    return json_create(buffer);             // 4. 문자열을 JSON으로 파싱
}
```

동작 순서는 다음과 같다.

1. `fopen` 으로 파일을 연다.
2. `fseek(fp, 0, SEEK_END)` 로 파일 끝으로 이동한 다음 `ftell(fp)` 로 현재 위치(= 파일 전체 크기)를 알아낸다. 이후 `SEEK_SET` 으로 다시 처음 위치로 되돌린다.
3. 그 크기만큼 `malloc` 으로 메모리를 할당받고, `fread` 로 파일 내용을 통째로 읽어 문자열에 담는다.
4. 완성된 문자열을 원래 `json_c.c` 에 구현되어 있던 `json_create()` 에 넘겨서 실제 JSON 파싱을 진행한다.

즉 `json_read()` 는 선언만 존재했고 구현이 없었으며, `my_json_read()` 는 그 빈자리를 직접 채운 함수이다.

코드 흐름 요약 (구현)

- `ast.json` 파일을 읽어서 문자열 형태로 저장한 뒤 JSON으로 파싱한다.
- 먼저 `FuncDef` 노드를 찾아 함수 정의를 확인한 뒤, 그 안에 있는 `decl` 에서 함수 이름과 타입 정보를 추출한다.

```
FuncDef
├── decl
│   ├── name (함수 이름)
│   └── type (반환 타입 / 매개변수 체인의 시작점)
```

- 반환 타입은 여러 단계의 노드를 따라가면서 실제 타입 이름을 찾는다.
- 함수 내부를 재귀적으로 탐색하여 `if` 문의 개수를 센 뒤, 함수 이름, 반환 타입, 매개변수 개수, `if` 문 개수를 출력한다.

심화 - 원본 코드 복원

`ast.json`에는 `BinaryOp`, `While`, `ArrayRef` 등 다양한 노드가 있어 모든 코드를 복원하기에는 범위가 너무 넓었다.

그래서 이번 과제에서는 비교적 구조가 단순한 함수 3개만 복원해 보았다.

선택한 함수: `main`, `error`, `be_push`

구현 방법

함수 본문(body) 안에 있는 문장들을 하나씩 읽어서 다시 C 문법으로 출력하도록 구현했다.

이번 과제에서는 다음 노드들만 처리했다.

- `Return` → `return 식;`
- `FuncCall` → `함수이름(인자들);`
- `ID` → 변수 이름 그대로 출력
- `Constant` → 값 그대로 출력
- `Compound` → 내부 문장을 하나씩 재귀적으로 출력

식(Expression)은 `render_expr()` 에서 문자열로 만들고, 문장(Statement)은 `render_stmt()` 에서 C 코드 형태로 변환했다. `Compound` 노드를 만나면 내부 문장들을 다시 같은 방식으로 처리하도록 재귀 호출을 사용했다.

실제 예시 - `error` 함수

`error` 함수의 `ast.json` 일부는 다음과 같다.

```
"body": {
  "_nodetype": "Compound",
  "block_items": [
    {
      "_nodetype": "FuncCall",
      "name": { "_nodetype": "ID", "name": "exit" },
      "args": {
        "_nodetype": "ExprList",
        "exprs": [
          { "_nodetype": "Constant", "type": "int", "value": "1" }
        ]
      }
    }
  ]
}
```

1단계 - `render_stmt()` 호출

`body`의 `_nodetype` 이 `Compound` 이므로 `{` 를 출력한 뒤, `** block_items **`에 있는 문장들을 하나씩 `** render_stmt() **`로 처리한다.

2단계 - `FuncCall` 처리

`block_items[0]` 의 `_nodetype` 이 `FuncCall` 이므로 `render_expr()` 를 호출한다.

3단계 - `render_expr()` 처리

- `name.name` 에서 함수 이름 `exit` 을 가져온다.
- `args.exprs` 를 순회하면서 인자를 처리한다.
- 인자는 `Constant` 노드이므로 값 `1` 을 그대로 사용한다.
- 최종적으로 `exit(1)` 문자열을 만든다.

4단계 - 문장 완성

`render_stmt()` 로 돌아와 끝에 `;` 를 붙여 `exit(1);` 를 출력한다.

5단계 - Compound 종료

모든 문장을 출력한 뒤 `}` 를 출력한다.

실행 결과

기본 결과

```

swimm@SWIM UCRT64 /c/Users/swimm/Desktop/School/White-hat/Assignment
$ ./ast
[1] int main()
    if문 개 수 : 0
[2] char * my_realloc(char * old, int oldlen, int newlen)
    if문 개 수 : 0
[3] void error()
    if문 개 수 : 0
[4] void takechar()
    if문 개 수 : 1
[5] void get_token()
    if문 개 수 : 7
[6] int peek(char * s)
    if문 개 수 : 0
[7] int accept(char * s)
    if문 개 수 : 1
[8] void expect(char * s)
    if문 개 수 : 1
[9] void save_int(char * p, int n)
    if문 개 수 : 0
[10] int load_int(char * p)
    if문 개 수 : 0
[11] void emit(int n, char * s)
    if문 개 수 : 1
[12] void be_push()
    if문 개 수 : 0
[13] void be_pop(int n)
    if문 개 수 : 0
[14] int sym_lookup(char * s)
    if문 개 수 : 1
[15] void sym_declare(char * s, int type, int value)
    if문 개 수 : 1
[16] int sym_declare_global(char * s)
    if문 개 수 : 1
[17] void sym_define_global(int current_symbol)
    if문 개 수 : 1
[18] void sym_get_value(char * s)
    if문 개 수 : 5
[19] void be_start()
    if문 개 수 : 0
[20] void be_finish()
    if문 개 수 : 0
[21] void promote(int type)
    if문 개 수 : 2
[22] int primary_expr()
    if문 개 수 : 9
[23] void binary1(int type)
    if문 개 수 : 0
[24] int binary2(int type, int n, char * s)
    if문 개 수 : 0
[25] int postfix_expr()
    if문 개 수 : 3
[26] int additive_expr()
    if문 개 수 : 2
[27] int shift_expr()
    if문 개 수 : 2
[28] int relational_expr()
    if문 개 수 : 0
[29] int equality_expr()
    if문 개 수 : 2
[30] int bitwise_and_expr()
    if문 개 수 : 0
[31] int bitwise_or_expr()
    if문 개 수 : 0
[32] int expression()
    if문 개 수 : 2
[33] void type_name()
    if문 개 수 : 0
[34] void statement()
    if문 개 수 : 8
[35] void program()

```

```

    if문 개 수 : 0
[34] void statement()
    if문 개 수 : 8
[35] void program()
    if문 개 수 : 4
[36] int main1()
    if문 개 수 : 0

총 함수 개 수 : 36

=== 심 화 : 원 본 소 스 코 드 복 원 (함 수 3개 ) ===

int main() {
    return main1();
}

void error() {
    exit(1);
}

void be_push() {
    emit(1, "\\x50");
}

swimm@SWIM UCRT64 /c/Users/swimm/Desktop/School/White-hat/Assignment
$

```

총 함수 개수: 36개

심화 결과

복원 결과는 `target.c` 의 해당 함수와 동일한 동작을 하는 C 코드 형태로 출력되었다. 다만 AST에는 주석, 공백, 들여쓰기 같은 서식 정보는 저장되지 않기 때문에 원본과 완전히 같은 형태로 복원되지는 않는다.

어려웠던 부분

1. gcc가 설치되어 있지 않았다

PowerShell에서 `gcc --version` 을 입력하자 명령어를 인식하지 못한다는 메시지가 출력되었다. 확인해보니 C언어 컴파일러 자체가 설치되어 있지 않았던 것이었다. MSYS2를 설치하여 UCRT64 터미널에서 `pacman -S mingw-w64-ucrt-x86_64-gcc` 를 입력하여 해결하였다.

2. 한글이 계속 깨짐

실행하면 `if문 개수: 0` 가 깨진 형태로 출력되었다. 결과값(숫자)은 정확했지만 한글만 깨졌다. 처음에는 VS Code 인코딩 문제로 판단해 EUC-KR로 다시 열어보거나 UTF-8로 저장해보았지만 해결되지 않았다. 확인해본 결과 파일 자체는 UTF-8이 맞았고(`file ast.c` 로 확

인함), 실제 원인은 Windows 콘솔이 출력 시 UTF-8을 사용하지 않는 것이었다. `#include <windows.h>` 를 추가하고 `main()` 첫 줄에 `SetConsoleOutputCP(CP_UTF8);` 을 추가하여 해결하였다.

3. 코드를 붙여넣는 과정에서 중간이 잘림

한글 문제를 해결한 후 다시 컴파일하자 `error: expected declaration or statement at end of input` 이 발생하였다. `tail -20 ast.c` 로 확인해보니 `count_if_recursive()` 함수 끝부분에서 파일이 잘려 있었다 (main 함수가 통째로 사라진 상태였다). 복사하는 과정에서 끝부분이 잘렸던 것이었으며, 파일 전체를 다시 지우고 처음부터 다시 붙여넣어 해결하였다.

질문 (마지막장용)

이번 과제를 진행하면서 AST만으로는 주석이나 들여쓰기 같은 정보를 복원할 수 없다는 점을 알게 되었습니다. 실제 디컴파일러는 이런 정보 손실을 어떤 방식으로 처리하나요?